Time-Series Visualization Critique Assignment

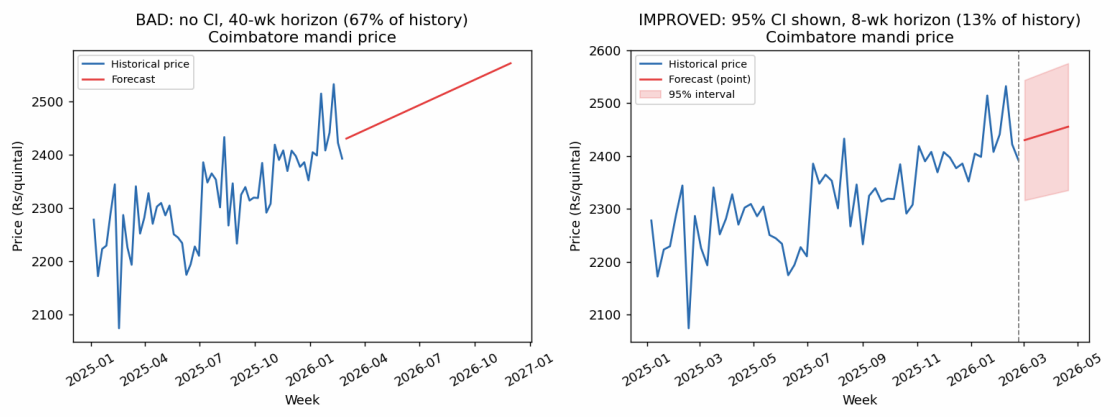
Roll No: 192324066 | Name: BOJJIREDDY VARUN KUMAR REDDY | Total: 50 Marks

Q1. Forecasting Trends

Dataset: 60 weeks (Jan-2025 to Feb-2026) of simulated weekly mandi price (Rs/quintal) for 7 Tamil Nadu districts (linear trend + 13-week seasonal cycle + noise). The Coimbatore series is used below to illustrate the forecast.

Code

```
coef = np.polyfit(t, y, 1)                    # trend fit on 60-wk history
# BAD: horizon = 40 weeks (67% of history), no interval
y_fore_bad = np.polyval(coef, np.arange(60, 100))
# GOOD: horizon = 8 weeks (13% of history), widening 95% CI
resid_std = np.std(y - np.polyval(coef, t))       # = 57.5 Rs/quintal
se = resid_std * np.sqrt(1 + h / n_hist)
ci95 = 1.96 * se
```



Evaluation

Score: 3 / 10

The bad chart shows a single deterministic forecast line with no visual sign that it carries uncertainty, and it extrapolates a straight-line trend 40 weeks ahead - 67% of the 60-week history used to fit it. For a weekly commodity-price series with a detrended residual standard deviation of ~57.5 Rs/quintal and a visible 13-week seasonal cycle the linear trend ignores, this is an unsupportable amount of extrapolation presented with false confidence.

Two specific weaknesses

1. No uncertainty band at all: even a modest 8-week-ahead 95% interval (using the fitted residual noise) already spans about ±120 Rs/quintal (~5% of the forecast level) - none of that risk is visible to the Coop President.

2. Horizon of 40 weeks (67% of the 60-week history) is far beyond what noisy weekly data can support, especially with a 13-week seasonal cycle in the data that the linear trend never models.

How the improved chart fixes this

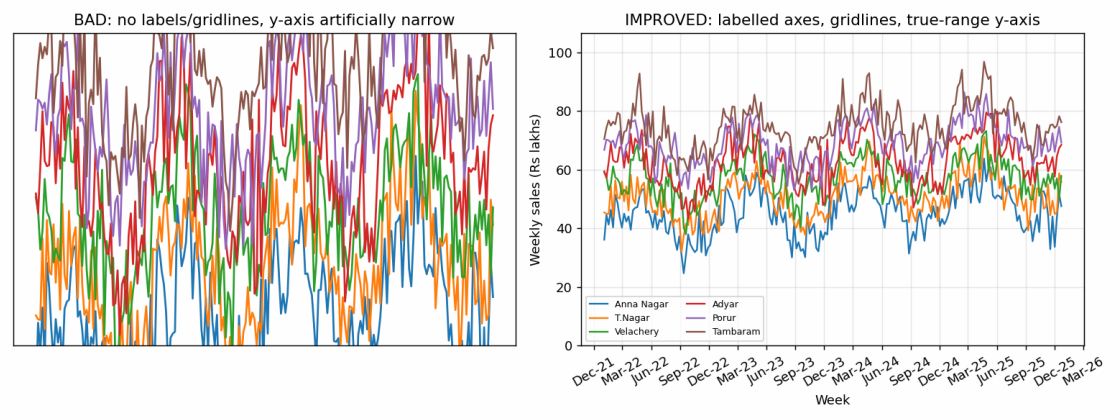
The horizon is cut to 8 weeks (13% of history) and a 95% prediction interval is shaded around the point forecast, widening with the horizon - giving an honest read of both the expected price and the uncertainty around it.

Q2. Individual Time Series

Dataset: 208 weeks (Jan-2022 to Dec-2025) of simulated weekly sales (Rs lakhs) for 6 store branches (trend + 52-week and 13-week seasonal components + noise).

Code

```
# BAD
ax.set_ylim(mean - 18, mean + 18)      # arbitrary narrow window
ax.set_xticks([]); ax.set_yticks([])   # no labels/gridlines
# GOOD
ax.set_ylim(0, values.max() * 1.1)     # true, zero-based range
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b-%y'))
ax.grid(alpha=0.3); ax.legend(ncol=2)
```



Evaluation

Score: 2 / 10

The chart has no axis ticks, labels or gridlines, so a viewer cannot recover a specific date or a specific rupee value from it. The y-axis window (36 units) is also roughly half the true 72.3-lakh range across branches (24.6-96.9), clipping genuine peaks/troughs and distorting the apparent volatility of each line.

Two specific weaknesses

1. Missing axis labels, tick marks and gridlines - the Sales Director cannot read off 'week X, branch Y sold Z lakhs' from the chart, defeating the basic purpose of the visualization.

2. y-axis compressed to 36 units against a true 72.3-lakh range (24.6 to 96.9) - several branches' data is clipped outside the visible window, so the perceived variability is an artefact of axis choice, not the underlying data.

How the improved chart fixes this

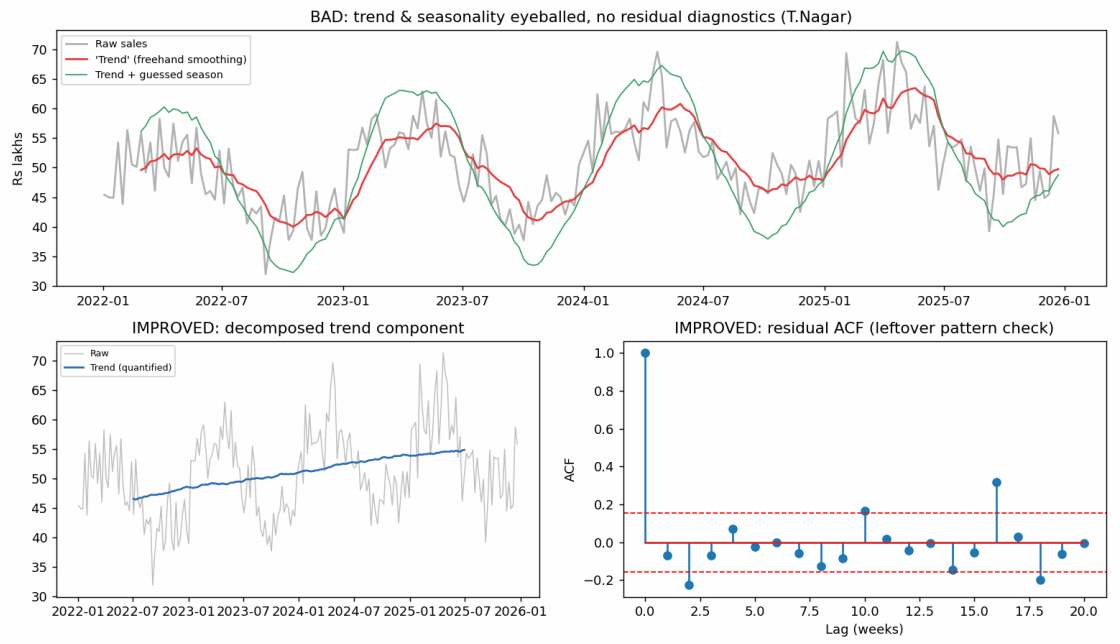
The improved chart uses a zero-based, full-range y-axis, monthly date ticks, gridlines and a legend, so both absolute sales levels and true week-to-week variability are honestly and precisely readable.

Q3. Seasonal Decomposition (STL)

Dataset: T.Nagar branch, 208 weeks. The true generating process has a dominant 52-week seasonal cycle (amplitude 8) plus a smaller 13-week sub-cycle (amplitude 3) on a rising trend.

Code

```
# BAD: freehand smoothing relabelled as 'trend'; seasonal amplitude guessed
eyeball_trend = moving_average(y, 9)              # window undisclosed
eyeball_seasonal = 8 * np.sin(2*np.pi*t/52)       # amplitude asserted, not fitted
# GOOD: decomposition + residual ACF check
trend, seasonal, resid = classical_decompose(y, period=52)
acf_vals = acf(resid, nlags=20)                   # flags leftover pattern
```



Evaluation

Score: 4 / 10

The bad chart is visually plausible but neither its 'trend' (a 9-week moving average with an undisclosed window) nor its seasonal amplitude (asserted as ±8 by eye) is actually estimated from the data, and no residual check was ever performed. The data-driven seasonal component actually swings from -10.4 to +13.8 Rs lakhs - larger and asymmetric versus the flat guess - and the residual ACF flags a real signal at lag 16 (ACF = 0.32, outside the ±0.16 95% band) that eyeballing could never reveal.

Two specific weaknesses

1. Seasonal amplitude was a flat guess (±8) rather than measured; the quantified seasonal component ranges from -10.4 to +13.8 Rs lakhs, understating the upswing by nearly 75%.

2. No residual diagnostic was performed, so the leftover ~13-16 week sub-cycle (residual ACF spike of 0.32 at lag 16, outside the significance band) went completely undetected.

How the improved chart fixes this

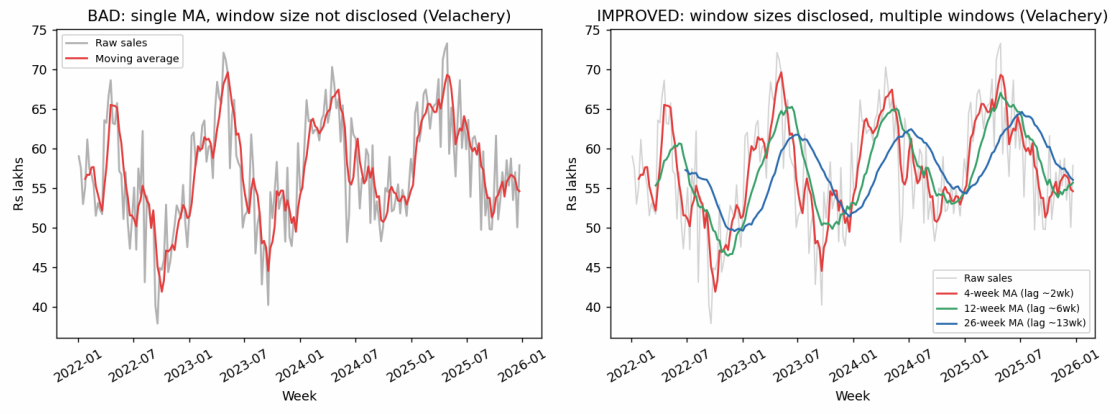
The improved panels separate and numerically quantify the trend and seasonal components from the data itself, then explicitly plot the residual ACF, catching the unmodelled sub-cycle that eyeballing could never surface.

Q4. Moving Averages

Dataset: Velachery branch, 208 weeks, same generating process as Q2/Q3.

Code

```
# BAD
ma = df.rolling(4).mean()                    # window never shown on chart
ax.plot(x, ma, label='Moving average')
# GOOD
for w in [4, 12, 26]:
    ax.plot(x, df.rolling(w).mean(), label=f'{w}-week MA (lag ~{w//2}wk)')
```



Evaluation

Score: 4 / 10

A 4-week moving average is a reasonable smoothing choice, but the chart never discloses the window size - the legend just says 'Moving average' - so the viewer cannot judge how much lag or smoothing is being applied, and a single window cannot serve both a fast reorder-timing decision and a slow strategic-trend read.

Two specific weaknesses

1. Window size (4 weeks) is never stated on the chart, so its effective lag (~2 weeks) and smoothing strength are invisible to the reader making a decision from it.

2. Only one window is shown; a longer window needed for strategic reading lags real turning points substantially - the 26-week average in this data does not register the raw peak of 18-Apr-2022 until 27-Jun-2022, a 10-week lag that would be far too slow for an operational (reordering) decision.

How the improved chart fixes this

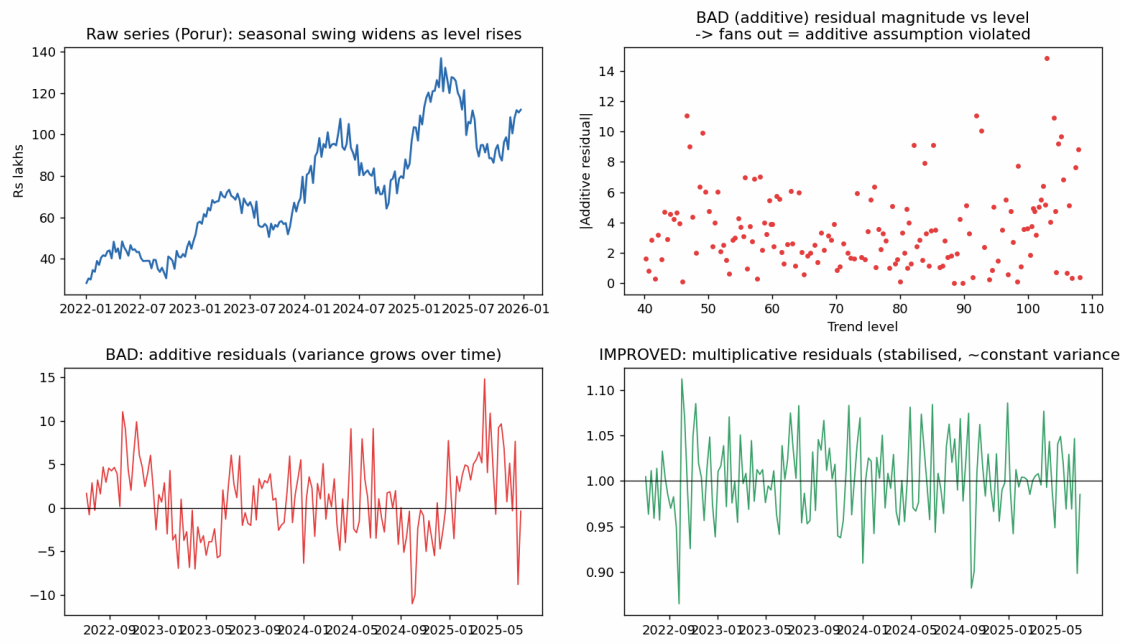
The improved chart labels every window with its approximate lag and overlays three windows (4/12/26 weeks) side by side, letting the viewer choose the right responsiveness-versus-smoothness trade-off for their specific decision.

Q5. Time-Series Decomposition

Dataset: Porur branch, 208 weeks; trend rises from ~30 to ~120 Rs lakhs while both the seasonal swing and the noise scale with the trend level (percentage-style growth).

Code

```
# BAD: additive decomposition applied unconditionally
trend, seasonal, resid_add = classical_decompose(y, 52, 'additive')
# GOOD: check residual magnitude vs level, then compare to multiplicative
trend_m, seasonal_m, resid_mul = classical_decompose(y, 52, 'multiplicative')
corr = np.corrcoef(trend[mask], np.abs(resid_add[mask]))[0, 1]   # = 0.11, tercile fan-out
```



Evaluation

Score: 3 / 10

The additive decomposition was applied without ever checking whether it fits. Splitting the additive residuals into low- and high-trend-level terciles shows their spread growing from 2.35 to 3.36 (+43%) - a textbook heteroscedasticity signature that an additive model cannot absorb. On the same data, the multiplicative decomposition produces near-identical residual spread across the two terciles (0.028 vs 0.028), confirming it honestly fits the variance behaviour.

Two specific weaknesses

1. No evidence was gathered before choosing additive over multiplicative; the residual-vs-level scatter (built only in the improved version) visibly fans out, and the |residual| standard deviation rises 43% from the lowest to the highest trend-level tercile.

2. The additive residual time series itself trends toward larger swings as the series grows, directly violating the additive model's implicit assumption of constant absolute noise.

How the improved chart fixes this

The improved analysis plots residual magnitude against trend level and compares additive vs multiplicative residual behaviour side by side; the multiplicative residuals are visibly and quantifiably more stable, giving objective evidence for the model choice rather than an unchecked assumption.

Appendix: Full Analysis Code

Complete Python script used to simulate the data and generate every chart above (matplotlib/pandas/numpy).

```
192324066_BOJJIREDDY VARUN KUMAR REDDY
Time Series Visualization Critique Assignment
Generates synthetic data + bad/improved chart pairs for Q1-Q5.
"""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
np.random.seed(66)
plt.rcParams["figure.dpi"] = 130
OUT = "/home/claude/work/"
# ---------------------------------------------------------------
# Shared helpers
# ---------------------------------------------------------------
def moving_average(x, window):
    return pd.Series(x).rolling(window, center=False).mean().values
def classical_decompose(x, period, model="additive"):
    """Simple classical decomposition: trend via centered MA, seasonal via
    period-wise averaging of detrended series, residual = leftover."""
    s = pd.Series(x)
    trend = s.rolling(period, center=True).mean()
    if model == "additive":
        detrended = s - trend
    else:
        detrended = s / trend
    seas_index = np.arange(len(s)) % period
    seasonal_avg = pd.Series(detrended).groupby(seas_index).mean()
    # normalize seasonal component
    if model == "additive":
        seasonal_avg = seasonal_avg - seasonal_avg.mean()
    else:
        seasonal_avg = seasonal_avg / seasonal_avg.mean()
    seasonal = seas_index.astype(float)
    seasonal = pd.Series(seasonal).map(seasonal_avg).values
    if model == "additive":
        resid = s.values - trend.values - seasonal
    else:
        resid = s.values / (trend.values * seasonal)
    return trend.values, seasonal, resid
def acf(x, nlags=20):
    x = np.asarray(x)
    x = x[~np.isnan(x)]
    x = x - x.mean()
    n = len(x)
    denom = np.sum(x**2)
    return np.array([np.sum(x[:n-k]*x[k:])/denom for k in range(nlags+1)])
# ===============================================================
# Q1: FORECASTING TRENDS  -- weekly mandi price (Rs/quintal), 7 districts
# ===============================================================
n_hist = 60
weeks1 = pd.date_range("2025-01-06", periods=n_hist, freq="W-MON")
districts = ["Coimbatore","Salem","Madurai","Trichy","Erode","Vellore","Thanjavur"]
price_data = {}
for i, d in enumerate(districts):
    base = 2200 + i*80
    trend = np.linspace(0, 220, n_hist)
    season = 60*np.sin(2*np.pi*np.arange(n_hist)/13)
    noise = np.random.normal(0, 55, n_hist)
    price_data[d] = base + trend + season + noise
df1 = pd.DataFrame(price_data, index=weeks1)
# focus district for forecast illustration
focus = "Coimbatore"
y = df1[focus].values
t = np.arange(n_hist)
# --- BAD: naive linear extrapolation, no CI, horizon = 40 weeks (~67% of history) ---
coef = np.polyfit(t, y, 1)
bad_horizon = 40
t_fore_bad = np.arange(n_hist, n_hist+bad_horizon)
y_fore_bad = np.polyval(coef, t_fore_bad)
weeks_fore_bad = pd.date_range(weeks1[-1]+pd.Timedelta(weeks=1), periods=bad_horizon, freq="W-MON")
# --- GOOD: short, defensible horizon (8 weeks, ~13% of history) with widening 95% CI ---
good_horizon = 8
resid_std = np.std(y - np.polyval(coef, t))
t_fore_good = np.arange(n_hist, n_hist+good_horizon)
y_fore_good = np.polyval(coef, t_fore_good)
weeks_fore_good = pd.date_range(weeks1[-1]+pd.Timedelta(weeks=1), periods=good_horizon, freq="W-MON")
h = np.arange(1, good_horizon+1)
se = resid_std*np.sqrt(1 + h/n_hist)          # widening interval with horizon
ci95 = 1.96*se
      f"week-8 forecast 95% CI half-width = {ci95[-1]:.1f} Rs/quintal "
      f"({100*ci95[-1]/y_fore_good[-1]:.1f}% of forecast level)")
fig, axes = plt.subplots(1, 2, figsize=(12,4.5))
ax = axes[0]
ax.plot(weeks1, y, color="#2b6cb0", label="Historical price")
ax.plot(weeks_fore_bad, y_fore_bad, color="#e53e3e", label="Forecast")
ax.set_title(f"BAD: no CI, {bad_horizon}-wk horizon (67% of history)\n{focus} mandi price")
ax.set_xlabel("Week"); ax.set_ylabel("Price (Rs/quintal)")
ax.legend(fontsize=8)
ax = axes[1]
ax.plot(weeks1, y, color="#2b6cb0", label="Historical price")
ax.plot(weeks_fore_good, y_fore_good, color="#e53e3e", label="Forecast (point)")
ax.fill_between(weeks_fore_good, y_fore_good-ci95, y_fore_good+ci95,
                color="#e53e3e", alpha=0.2, label="95% interval")
ax.axvline(weeks1[-1], color="gray", ls="--", lw=1)
ax.set_title(f"IMPROVED: 95% CI shown, {good_horizon}-wk horizon (13% of history)\n{focus} mandi price")
ax.set_xlabel("Week"); ax.set_ylabel("Price (Rs/quintal)")
ax.legend(fontsize=8)
for a in axes:
    a.tick_params(axis='x', rotation=30)
plt.tight_layout()
plt.savefig(OUT+"q1_forecast.png")
plt.close()
# ===============================================================
# Q2: INDIVIDUAL TIME SERIES -- weekly sales (Rs lakhs), 6 branches
# ===============================================================
n2 = 208   # 4 years of weekly data -> enough seasonal cycles for a stable decomposition
weeks2 = pd.date_range("2022-01-03", periods=n2, freq="W-MON")
branches = ["Anna Nagar","T.Nagar","Velachery","Adyar","Porur","Tambaram"]
sales_data = {}
for i, b in enumerate(branches):
```

```
    base = 40 + i*6
    trend = np.linspace(0, 10, n2)
    season = 8*np.sin(2*np.pi*np.arange(n2)/52) + 3*np.sin(2*np.pi*np.arange(n2)/13)
    noise = np.random.normal(0, 4, n2)
    sales_data[b] = base + trend + season + noise
df2 = pd.DataFrame(sales_data, index=weeks2)
fig, axes = plt.subplots(1, 2, figsize=(12,4.5))
ax = axes[0]
for b in branches:
    ax.plot(range(n2), df2[b].values)
ax.set_ylim(df2.values.mean()-18, df2.values.mean()+18)   # misleadingly narrow y-axis
ax.set_title("BAD: no labels/gridlines, y-axis artificially narrow")
ax.set_xticks([]); ax.set_yticks([])
ax = axes[1]
for b in branches:
    ax.plot(weeks2, df2[b].values, label=b, linewidth=1.3)
ax.set_ylim(0, df2.values.max()*1.1)                    # honest, zero-based range
ax.set_title("IMPROVED: labelled axes, gridlines, true-range y-axis")
ax.set_xlabel("Week"); ax.set_ylabel("Weekly sales (Rs lakhs)")
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b-%y"))
ax.grid(alpha=0.3)
ax.legend(fontsize=7, ncol=2)
axes[1].tick_params(axis='x', rotation=30)
plt.tight_layout()
plt.savefig(OUT+"q2_individual.png")
plt.close()
    df2[branches[0]].iloc[:15].std(), df2[branches[0]].std())
    f"true span={df2.values.max()-df2.values.min():.1f}, BAD chart y-window span=36")
# ============================================================
# Q3: SEASONAL DECOMPOSITION (STL) -- one branch, weekly sales
# ============================================================
branch3 = "T.Nagar"
y3 = df2[branch3].values
period3 = 52
# --- BAD: eyeballed trend (single freehand smooth) + assumed seasonality, no residual check ---
eyeball_trend = moving_average(y3, 9)  # crude, arbitrary smoothing, not disclosed as such
eyeball_seasonal_amplitude = 8  # "guessed" amplitude from squinting at the plot
eyeball_seasonal = eyeball_seasonal_amplitude*np.sin(2*np.pi*np.arange(n2)/52)
fig = plt.figure(figsize=(12,7))
gs = fig.add_gridspec(2,2)
ax0 = fig.add_subplot(gs[0,:])
ax0.plot(weeks2, y3, color="gray", alpha=0.6, label="Raw sales")
ax0.plot(weeks2, eyeball_trend, color="#e53e3e", label="'Trend' (freehand smoothing)")
ax0.plot(weeks2, eyeball_trend+eyeball_seasonal, color="#38a169", lw=1, label="Trend + guessed season")
ax0.set_title(f"BAD: trend & seasonality eyeballed, no residual diagnostics ({branch3})")
ax0.legend(fontsize=8); ax0.set_ylabel("Rs lakhs")
trend3, seasonal3, resid3 = classical_decompose(y3, period3, "additive")
ax1 = fig.add_subplot(gs[1,0])
ax1.plot(weeks2, y3, color="gray", alpha=0.5, lw=0.8, label="Raw")
ax1.plot(weeks2, trend3, color="#2b6cb0", label="Trend (quantified)")
ax1.set_title("IMPROVED: decomposed trend component")
ax1.legend(fontsize=7)
ax2 = fig.add_subplot(gs[1,1])
lags = np.arange(21)
acf_vals = acf(resid3, 20)
ax2.stem(lags, acf_vals)
ci_bound = 1.96/np.sqrt(np.sum(~np.isnan(resid3)))
ax2.axhline(ci_bound, color="red", ls="--", lw=1)
ax2.axhline(-ci_bound, color="red", ls="--", lw=1)
ax2.set_title("IMPROVED: residual ACF (leftover pattern check)")
ax2.set_xlabel("Lag (weeks)"); ax2.set_ylabel("ACF")
plt.tight_layout()
plt.savefig(OUT+"q3_stl.png")
plt.close()
max_acf_outside_band = np.max(np.abs(acf_vals[1:])[np.abs(acf_vals[1:])>ci_bound]) if np.any(np.abs(acf_vals[1:])>ci_bound) else 0
worst_lag = int(np.argmax(np.abs(acf_vals[1:])>ci_bound)+1) if np.any(np.abs(acf_vals[1:])>ci_bound) else None
    "worst lag:", worst_lag, "value:", acf_vals[worst_lag] if worst_lag else None)
    f"actual quantified seasonal component range = [{np.nanmin(seasonal3):.1f}, {np.nanmax(seasonal3):.1f}] Rs lakhs")
# ============================================================
# Q4: MOVING AVERAGES -- one branch, weekly sales
# ============================================================
branch4 = "Velachery"
y4 = df2[branch4].values
ma_bad = moving_average(y4, 4)  # window not disclosed to viewer in bad chart
fig, axes = plt.subplots(1, 2, figsize=(12,4.5))
ax = axes[0]
ax.plot(weeks2, y4, color="gray", alpha=0.6, label="Raw sales")
ax.plot(weeks2, ma_bad, color="#e53e3e", label="Moving average")  # no window size in label
ax.set_title(f"BAD: single MA, window size not disclosed ({branch4})")
ax.set_xlabel("Week"); ax.set_ylabel("Rs lakhs"); ax.legend(fontsize=8)
ax.tick_params(axis='x', rotation=30)
ax = axes[1]
ax.plot(weeks2, y4, color="lightgray", lw=1, label="Raw sales")
for w, c in [(4, "#e53e3e"), (12, "#38a169"), (26, "#2b6cb0")]:
    ax.plot(weeks2, moving_average(y4, w), color=c, label=f"{w}-week MA (lag ~{w//2}wk)")
ax.set_title(f"IMPROVED: window sizes disclosed, multiple windows ({branch4})")
ax.set_xlabel("Week"); ax.set_ylabel("Rs lakhs"); ax.legend(fontsize=7.5)
ax.tick_params(axis='x', rotation=30)
plt.tight_layout()
plt.savefig(OUT+"q4_moving_avg.png")
plt.close()
# correlation of MA(4) vs raw turning points as lag illustration
peak_raw = weeks2[np.argmax(y4[:52])]
peak_ma4 = weeks2[np.nanargmax(ma_bad[:52])]
peak_ma26 = weeks2[np.nanargmax(moving_average(y4,26)[:52])]
lag26_weeks = (peak_ma26 - peak_raw).days // 7
    f"-> 26-wk MA lags true peak by {lag26_weeks} weeks")
# ============================================================
# Q5: TIME-SERIES DECOMPOSITION -- additive vs multiplicative, one branch
# ============================================================
branch5 = "Porur"
n5 = n2
trend5 = 30 + np.linspace(0, 90, n5)         # strong growing level
season_pct = 0.22*np.sin(2*np.pi*np.arange(n5)/52)   # seasonal swing scales with level (%)
noise5 = np.random.normal(0, 1, n5)*(0.4 + trend5/25)   # noise also grows with level
y5 = trend5*(1+season_pct) + noise5
trend_add, seasonal_add, resid_add = classical_decompose(y5, 52, "additive")
trend_mul, seasonal_mul, resid_mul = classical_decompose(y5, 52, "multiplicative")
fig, axes = plt.subplots(2, 2, figsize=(12,7))
axes[0,0].plot(weeks2, y5, color="#2b6cb0")
axes[0,0].set_title(f"Raw series ({branch5}): seasonal swing widens as level rises")
axes[0,0].set_ylabel("Rs lakhs")
axes[0,1].scatter(trend_add, np.abs(resid_add), s=8, color="#e53e3e")
axes[0,1].set_title("BAD (additive) residual magnitude vs level\n-> fans out = additive assumption violated")
axes[0,1].set_xlabel("Trend level"); axes[0,1].set_ylabel("|Additive residual|")
axes[1,0].plot(weeks2, resid_add, color="#e53e3e", lw=1)
axes[1,0].axhline(0, color="black", lw=0.7)
axes[1,0].set_title("BAD: additive residuals (variance grows over time)")
axes[1,1].plot(weeks2, resid_mul, color="#38a169", lw=1)
axes[1,1].axhline(1, color="black", lw=0.7)
```

```
axes[1,1].set_title("IMPROVED: multiplicative residuals (stabilised, ~constant variance)")
plt.tight_layout()
plt.savefig(OUT+"q5_decomposition.png")
plt.close()
# quantify heteroscedasticity: correlation between trend level and |residual|,
# and split by low/high trend-level tercile (robust to NaN edges from centered MA)
mask = ~np.isnan(trend_add) & ~np.isnan(resid_add)
corr_add = np.corrcoef(trend_add[mask], np.abs(resid_add[mask]))[0,1]
mask_m = ~np.isnan(trend_mul) & ~np.isnan(resid_mul)
corr_mul = np.corrcoef(trend_mul[mask_m], np.abs(resid_mul[mask_m]-1))[0,1]
lvl = trend_add[mask]; res = np.abs(resid_add[mask])
lo = res[lvl <= np.percentile(lvl,33)]; hi = res[lvl >= np.percentile(lvl,67)]
    f"std|resid| low-level tercile={lo.std():.2f} vs high-level tercile={hi.std():.2f}")
lvl_m = trend_mul[mask_m]; res_m = np.abs(resid_mul[mask_m]-1)
lo_m = res_m[lvl_m <= np.percentile(lvl_m,33)]; hi_m = res_m[lvl_m >= np.percentile(lvl_m,67)]
    f"std|resid%| low-level tercile={lo_m.std():.3f} vs high-level tercile={hi_m.std():.3f}")
```